

Acoustic Modem - Arınç Demir - İrem Önen

1. High-Level Architecture

The goal is to create a half-duplex (one talks, the other listens) acoustic modem in Python. We will convert text into audio using **Frequency Shift Keying (FSK)** and structure the data using an **asynchronous UART-like protocol**.

Half-Duplex Operation: If a computer transmits, its own microphone will hear its own speakers perfectly. To avoid writing complex echo-cancellation to ignore ourselves, we will enforce **Half-Duplex** behavior. When a user is transmitting, their local receiver is explicitly muted or ignored (like Walkie-Talkies).

The project breaks down into three main layers:

1. **Application Layer:** The UI/CLI where the user types text and sees received text.
2. **Data Link Layer (UART Framing):** Converting characters to bit streams (adding start and stop bits).
3. **Physical Layer (Audio FSK):** Converting bit streams into sine waves (Tx) and converting microphone audio back into bits (Rx).

2. The Data Link Layer (UART Protocol)

We will borrow heavily from standard UART principles.

- **Baud Rate & Combating Echoes:** 10 baud (bits per second). At 10 baud, each bit lasts exactly 100 milliseconds. This extremely slow rate is a deliberate design choice to combat **Inter-Symbol Interference**. In a physical room, sound bounces off walls causing echoes. The echo of a 3000Hz tone might overlap with a subsequent 2000Hz tone. At 100ms per bit, the room echoes from a previous bit will completely decay before we sample the middle of the next bit, ensuring a clean read.
- **Idle State:** Standard UART holds a "High" (1) state when idle. Emitting a continuous 3000 Hz tone will be incredibly annoying to humans. Instead, we should assume the medium is "Silent" when idle, and use a brief **Preamble** (a wake-up tone) before sending a frame.
- **Frame Structure:**
 - **Preamble:** ~200ms of 3000Hz (wakes up the receiver and provides a clear signal to lock onto).
 - **Start Bit:** 1 bit of 2000Hz (Logic 0). Tells the receiver "start the clock!".
 - **Data Bits:** 8 bits representing the ASCII character (Least Significant Bit first, standard UART).
 - **Stop Bit:** 1 bit of 3000Hz (Logic 1).
 - *Total:* 10 bits per character. At 10 baud, one character takes 1 second to transmit.

3. The Physical Layer (Modulation / Tx)

Goal: Take an array of bits and generate an audio waveform.

- **Frequencies:**
 1. Logic 0 = 2000 Hz
 2. Logic 1 = 3000 Hz
- **Tx Pipeline:**
 1. User types "Hello".
 2. Convert to ASCII binary: **01001000...**

3. Add framing: **[Preamble] 0 00010010 1 ...**
4. Map to frequencies.
5. Generate and concatenate discrete audio samples using NumPy.
6. Play audio buffer through speakers.

4. The Physical Layer (Demodulation / Rx)

Goal: Listen to the microphone, detect the tones, and reconstruct the bits.

- **Audio Capture:** Continuously record audio chunks in a background thread.
- **Tone Detection & Volume Sensitivity:** Because speakers vary in power and laptops will be placed at different distances, absolute volume thresholds would break easily. To remain immune to overall volume changes, we won't look at absolute volume. Instead, we will calculate the energy at both frequencies simultaneously (using FFT or Goertzel) and compare them:
 1. If **Energy(3000Hz) > Energy(2000Hz)**, record a **1**.
 2. If **Energy(2000Hz) > Energy(3000Hz)**, record a **0**.
- As long as the signal is louder than the background noise, the dominant frequency wins.
- **Preamble Detection (Ignoring Room Noise):** To distinguish our 3000Hz preamble from background silence, human speech, or a random chair squeak, we will implement three "squelch" mechanisms:
 1. **Signal-to-Noise Ratio (SNR):** Human voices are broadband, spanning many frequencies. Our modem is narrowband. We will verify that the 3000Hz energy constitutes the vast majority of the overall background noise energy before triggering.
 2. **Minimum Threshold (Noise Floor):** The 3000Hz tone must cross a hard, predetermined minimum amplitude floor to prevent the system from triggering on pure silence.
 3. **Time Lock:** A random whistle might hit 3000Hz for a split second. The receiver will require the 3000Hz tone to remain stable and dominant for a continuous duration (e.g., 100-150ms) before it formally "arms" itself to await the Start Bit.
- **Clock Synchronization (like UART):**
 1. The Rx is sitting idle, listening to background noise.
 2. It detects a strong 3000Hz signal (Preamble) that passes the squelch checks. It arms itself.
 3. It detects the transition to 2000Hz (The Start Bit).
 4. **The Clock Starts:** Once the start bit edge is found, the Rx waits exactly **1.5 * bit_duration** to sample the *middle* of the first data bit. Then it samples every **1.0 * bit_duration** after that to read the remaining 8 bits.
 5. Check the Stop Bit. If valid, append the character to the screen.

5. Tech Stack: Python Libraries

- **sounddevice:** For non-blocking, asynchronous audio playback and recording.
- **numpy:** For generating the sine waves (Tx) and managing audio arrays, and FFT.
- **threading:** To run the microphone listener loop simultaneously with the UI/keyboard input loop.